

软测第四次

```
d:\Desktop>pytest test_calculator.py
===== test session starts =====
platform win32 -- Python 3.11.7, pytest-7.4.0, pluggy-1.0.0
rootdir: d:\Desktop
plugins: cov-6.1.1, anyio-4.2.0
collected 4 items

test_calculator.py .... [100%]

===== 4 passed in 0.04s =====
```

```
命令提示符
D:\anaconda\Lib\site-packages\_pytest\python.py:617: in _importtestmodule
    mod = import_path(self.path, mode=importmode, root=self.config.rootpath)
D:\anaconda\Lib\site-packages\_pytest\pathlib.py:565: in import_path
    importlib.import_module(module_name)
D:\anaconda\Lib\importlib\__init__.py:126: in import_module
    return _bootstrap._gcd_import(name[level:], package, level)
<frozen importlib._bootstrap>:1204: in _gcd_import
    ???
<frozen importlib._bootstrap>:1176: in _find_and_load
    ???
<frozen importlib._bootstrap>:1147: in _find_and_load_unlocked
    ???
<frozen importlib._bootstrap>:690: in _load_unlocked
    ???
D:\anaconda\Lib\site-packages\_pytest\assertion\rewrite.py:169: in exec_module
    source_stat, co = _rewrite_test(fn, self.config)
D:\anaconda\Lib\site-packages\_pytest\assertion\rewrite.py:351: in _rewrite_test
    tree = ast.parse(source, filename=strfn)
D:\anaconda\Lib\ast.py:50: in parse
    return compile(source, filename, mode, flags,
           File "d:\Desktop\test_calculator.py", line 6
           return a - b
           ^
E     TabError: inconsistent use of tabs and spaces in indentation
===== short test summary info =====
ERROR test_calculator.py
!!!!!!!!!!!!!!!!!!!! Interrupted: 1 error during collection !!!!!!!!!!!!!!!!!!!!!
===== 1 error in 0.25s =====

d:\Desktop>
```

```
D:\Desktop\新建文件夹>pytest --cov=calculator --cov-report=html
===== test session starts =====
platform win32 -- Python 3.11.7, pytest-7.4.0, pluggy-1.0.0
rootdir: D:\Desktop\新建文件夹
plugins: cov-6.1.1, anyio-4.2.0
collected 4 items

test_calculator.py .... [100%]

===== tests coverage =====
coverage: platform win32, python 3.11.7-final-0

Coverage HTML written to dir htmlcov

===== 4 passed in 0.16s =====
```

Coverage report: 100%

Files Functions Classes

filter...



☐ hide covered

coverage.py v7.8.0, created at 2025-05-22 21:41 +0800

File ▲	class	statements	missing	excluded	coverage
calculator.py	Calculator	6	0	0	100%
calculator.py	(no class)	5	0	0	100%
Total		11	0	0	100%

coverage.py v7.8.0, created at 2025-05-22 21:41 +0800

Coverage report: 91%

Files Functions Classes

filter...



☐ hide covered

coverage.py v7.8.0, created at 2025-05-22 21:54 +0800

File ▲	class	statements	missing	excluded	coverage
calculator.py	Calculator	6	1	0	83%
calculator.py	(no class)	5	0	0	100%
Total		11	1	0	91%

coverage.py v7.8.0, created at 2025-05-22 21:54 +0800

Coverage report: 91%

Files Functions Classes

filter...



☐ hide covered

coverage.py v7.8.0, created at 2025-05-22 21:54 +0800

File ▲	statements	missing	excluded	coverage
calculator.py	11	1	0	91%
Total	11	1	0	91%

coverage.py v7.8.0, created at 2025-05-22 21:54 +0800

Coverage report: 91%

Files Functions Classes

filter...



☐ hide covered

coverage.py v7.8.0, created at 2025-05-22 21:54 +0800

File ▲	function	statements	missing	excluded	coverage
calculator.py	Calculator.add	1	1	0	0%
calculator.py	Calculator.subtract	1	0	0	100%
calculator.py	Calculator.multiply	1	0	0	100%
calculator.py	Calculator.divide	3	0	0	100%
calculator.py	(no function)	5	0	0	100%
Total		11	1	0	91%

coverage.py v7.8.0, created at 2025-05-22 21:54 +0800

```
D:\Desktop\新建文件夹>pytest test_todo.py
===== test session starts =====
platform win32 -- Python 3.11.7, pytest-7.4.0, pluggy-1.0.0
rootdir: D:\Desktop\新建文件夹
plugins: cov-6.1.1, anyio-4.2.0
collected 10 items

test_todo.py ..... [100%]

===== 10 passed in 0.05s =====
```

```
D:\Desktop\新建文件夹>pytest --cov=todo_list --cov-report=html
===== test session starts =====
platform win32 -- Python 3.11.7, pytest-7.4.0, pluggy-1.0.0
rootdir: D:\Desktop\新建文件夹
plugins: cov-6.1.1, anyio-4.2.0
collected 13 items

test_calculator.py ... [ 23%]
test_todo.py ..... [100%]

===== tests coverage =====
----- coverage: platform win32, python 3.11.7-final-0 -----

Coverage HTML written to dir htmlcov
===== 13 passed in 0.15s =====
```

Coverage for **todo_list.py**: 100%

32 statements 32 run 0 missing 0 excluded

[« prev](#) [^ index](#) [» next](#) coverage.py v7.8.0, created at 2025-05-22 22:52 +0800

```
1 class Todo:
2     def __init__(self, name, completed=False):
3         self.name = name
4         self.completed = completed
5
6     def mark_completed(self):
7         self.completed = True
8
9     def __str__(self):
10        return f"{self.name} - {'Completed' if self.completed else 'Pending'}"
11
12 class TodoList:
13     def __init__(self):
14         self.todos = []
15
16     def add_task(self, name):
17         if not name:
18             raise ValueError("Task name cannot be empty")
19         self.todos.append(Todo(name))
20
21     def remove_task(self, name):
22         task_to_remove = None
23         for task in self.todos:
24             if task.name == name:
25                 task_to_remove = task
26         if task_to_remove:
27             self.todos.remove(task_to_remove)
```

```
D:\Desktop\新建文件夹>pytest --cov=test_data_processor --cov-report=html
===== test session starts =====
platform win32 -- Python 3.11.7, pytest-7.4.0, pluggy-1.0.0
rootdir: D:\Desktop\新建文件夹
plugins: cov-6.1.1, mock-3.14.0, anyio-4.2.0
collected 15 items

test_calculator.py ... [ 20%]
test_data_processor.py .. [ 33%]
test_todo.py ..... [100%]

===== tests coverage =====
----- coverage: platform win32, python 3.11.7-final-0 -----

Coverage HTML written to dir htmlcov
===== 15 passed in 0.43s =====
```

Coverage for **test_data_processor.py**: 100%

25 statements

25 run

0 missing

0 excluded

« prev ^ index » next coverage.py v7.8.0, created at 2025-05-22 23:23 +0800

```
1 # test_data_processor.py
2 import pytest
3 from calculator import Calculator
4 from data_processor import DataProcessor
5
6 @pytest.fixture
7 def calculator():
8     """Fixture: 提供Calculator实例"""
9     return Calculator()
10
11 @pytest.fixture
12 def data_processor(calculator):
13     """Fixture: 提供DataProcessor实例"""
14     return DataProcessor(calculator)
15
16 def test_process_data_success(mock, data_processor):
17     """
18     测试process_data方法在成功获取数据时的行为
19     """
20     # Mock requests.get方法
21     mock_get = mock.patch('data_processor.requests.get')
22     mock_response = mock.MagicMock()
23     mock_response.status_code = 200
24     mock_response.json.return_value = [1, 2, 3, 4]
25     mock_get.return_value = mock_response
26
```